

GuardianIQ Phase 2: RAG Compliance & QA Report

Document Reference: GIQ-P2-RAG-REP-001
Date of Report: June 25, 2026
Project Phase: Phase 2 Governed Workflow Scheduling & AI Agent Execution
Target Build: Phase 2 End-to-End Workflow Scheduling and Execution
Audit Target Path: C:\Users\ayus\Desktop\GuardianIQ--1\docs
QA Status: [AMBER] (Passed with Gaps & Critical Bugs)

1. Executive Summary

This report evaluates the current codebase implementation against the specifications outlined in the **GuardianIQ Phase 2 Spec Documents** ([GuardianIQ Phase2 WFSpecification.pdf](#) and [GuardianIQ Phase2 Governed Workflow Scheduling AI Agent Execution Developer Spec.pdf](#)).

The overall system is functionally complete and demonstrates: - **Backend Stability:** All 26 unit and integration tests compile and pass successfully. - **Frontend Build:** The React frontend compiles with no TypeScript errors (`npm run typecheck` is successful) and bundles cleanly. - **Critical Issues:** Two major compliance failures exist around API contract inconsistencies and missing ABAC/RBAC validation on several sensitive endpoints.

2. RAG Compliance Matrix

Project Scope Heading	RAG Status	Primary Finding / Discrepancy
1. Executive Summary & Objectives	[PASS]	Core capabilities (schedules, runs, steps, notifications, approvals) are fully implemented.
2. Scope and Boundaries	[PASS]	All in-scope items are functional; out-of-scope capabilities are correctly deferred.
3. End-to-End Workflow Lifecycle & State Machines	[PASS]	State transitions for schedules and runs are correctly enforced in the service layer.
4. Backend Services & Architecture	[PASS]	Services are modularly designed and aligned under <code>app/modules</code> .

5. Database Components & Schema	[PASS]	PostgreSQL tables and migrations match the target schema exactly with cascade deletes.
6. API Design & Contract Specification	[FAIL]	Non-compliant response envelopes, incorrect create schedule payload, and missing dynamic sorting.
7. Frontend Modules & Screen Design	[PASS]	React frontend typechecks successfully and contains all specified screen routes.
8. Scheduling & Execution Engine	[PASS]	Background worker polls active schedules correctly; SLA breaches and retries function as designed.
9. AI Agent Boundary Controls	[PASS]	Rejects tool calls outside the allowed tool list and triggers high-risk notifications.
10. Security & Authorization (RBAC + ABAC)	[FAIL]	Critical Security Gap: Sensitive endpoints (schedule update, submit, pause, run-now, run list/detail) lack permission checks.
11. Governance Event & Audit Logging	[AMBER]	Assignment creation and update write APIs fail to log events.
12. Notifications, Escalations & SLA	[PASS]	SLA breach alerts and high-risk findings successfully dispatch notifications.
13. Error Handling & Failure Recovery	[PASS]	Automatic retry loop handles transient errors correctly while blocking security/validation retries.

3. Critical Findings and Discrepancies

3.1. Missing Authorization Checks on Sensitive APIs

Severity: [CRITICAL] - **Issue:** The `AuthorizationDecisionService` is **not called** on multiple sensitive endpoints. RBAC decorators are used on write endpoints, but ABAC validations (owner, risk, department, mode boundaries) are missing from schedule updates (PUT), submissions, status changes (pause, resume, retire), manual triggers (`run-now`), and run list/detail queries, allowing any authenticated user to perform these actions.

- **Affected Endpoints:** - GET `/api/v1/workflow-scheduler/schedules` - GET

`/api/v1/workflow-scheduler/schedules/{id}` - PUT `/api/v1/workflow-scheduler/schedules/{id}` - POST

`/api/v1/workflow-scheduler/schedules/{id}/submit` - POST

`/api/v1/workflow-scheduler/schedules/{id}/pause` - POST

`/api/v1/workflow-scheduler/schedules/{id}/resume` - POST

`/api/v1/workflow-scheduler/schedules/{id}/retire` - POST

`/api/v1/workflow-scheduler/schedules/{id}/run-now` - GET `/api/v1/workflow-runs` - GET

`/api/v1/workflow-runs/{run_id}/steps` - **Suggestion:** Bind `AuthorizationDecisionService` checks inside all missing schedule mutation and workflow run query routes.

3.2. Inconsistent Response Envelope Structure

Severity: [HIGH] - **Issue:** Section 11.1 specifies: "All responses must use standard envelope: `success`, `data`, `error`, `request_id`, `timestamp`." Currently, standard validation handlers, exception middleware, and the authorization evaluate route return the standard app response wrapper (`status`, `request_id`, `message`, `data`, `error_code`) which lacks the required `success`, `error`, and `timestamp` fields. - **Affected Endpoints:** `/api/v1/authorization/evaluate` and all app-wide exceptions/validation errors. - **Suggestion:** Modify `ResponseStandardizationMiddleware` or update exception handlers in [exceptions.py](#) to guarantee that every response contains the Phase 2 envelope fields.

3.3. Create Schedule Payload Mismatch

Severity: [HIGH] - **Issue:** Section 11.4 lists the expected payload format of `data` on schedule creation to include `"schedule_id"`, `"schedule_status"`, `"approval_required"`, and `"approval_reason"`. Currently, the API returns the entire schedule database representation using key `"id"` instead of `"schedule_id"`, and completely lacks the `"approval_reason"` field. - **Suggestion:** Update the return serializer on the schedule creation API in [phase2_scheduler_routes.py](#) to match the expected format.

3.4. Missing Audit Events for Agent Assignments

Severity: [AMBER] MEDIUM - **Issue:** All write APIs must publish governance events. However, write APIs for agent assignments (`POST /schedules/{id}/agent-assignments` and `PUT`) do not invoke the `GovernanceEventService` to log events. - **Suggestion:** Inject `event_service.publish_event` inside the agent-assignments write routes.

3.5. Lack of Dynamic Sorting

Severity: **[AMBER] MEDIUM** - **Issue:** List APIs must support pagination, sorting, and filters. However, repository queries for schedules and runs hardcode their sorting ordering to descending creation time (`created_at DESC`), and there are no dynamic query parameter bindings for `sort_by` or `order` (e.g. `asc/desc`). - **Suggestion:** Expose `sort_by` and `order` query parameters in list routes and support dynamic `order_by` clauses in the repository queries.

4. QA Validation Summary

4.1. Backend Unit & Integration Tests

All tests in the `app/tests/` directory pass cleanly: - **Total Tests Run:** 26 - **Passed:** 26 - **Failures:** 0 - **Status:** **[PASS]**

4.2. Frontend Compile & Safety Check

Strict TypeScript compiler check runs with no errors or warnings: - **Command:** `npm run typecheck (tsc --noEmit)` - **Status:** **[PASS]**

4.3. Frontend Production Build

Vite production bundler builds assets with no blocking errors: - **Command:** `npm run build` - **Status:** **[PASS]**

5. QA Verdict and Sign-off

While the functional logic is sound and the test suites pass successfully, the security authorization gap (missing checks on sensitive endpoints) and contract non-compliances must be resolved before production deployment.

Final Status: **[AMBER] AMBER (Passed with Gaps)**